A BRIEF FOR THE FULL STACK ENGINEER ROLE

The Build *Exercise.*

A scoped, deployed, defensible system — built the way you'd actually build it, with the tools you'd actually use.

TIME	DEADLINE	TOOLS	SUBMIT TO
6 – 8 hrs	7 days	Anything	hello@t3c.ai
more is a signal we'd rather not see	from receipt of this brief	Claude Code, Cursor, Copilot — all expected	subject: FSE Build — [Your Name]

THE ASK

Build a small **warehouse management platform** with authentication, role-based access, multi-tenant data isolation, and a real analytics view powered by a Postgres → BigQuery pipeline. Deploy it on Google Cloud. Seed it with the data we specify. Walk us through your decisions.

This exercise mirrors the actual shape of the work. We are not testing Leetcode. We are testing whether you can ship a small system that works, makes sensible trade-offs on a real architecture, and that you can *explain*, end to end.

I. What to Build

A warehouse management platform with the four pieces below. Partial is fine. Polish is not the point — *thinking* is.

1 — Authentication & RBAC

- Login via **WorkOS AuthKit** — free tier is sufficient.
- Three roles with **genuinely enforced** permissions: *Admin*, *Warehouse Manager*, *Operator*.
- Enforcement lives at the **API and data layer** — never just hidden UI buttons. We will probe this in the walkthrough.

2 — Core Domain (Transactional)

All transactional reads and writes run against Cloud SQL Postgres via Prisma.

- **Warehouses** — name, location, capacity.
- **Inventory items** — SKU, name, quantity per warehouse.
- **Stock movements** — inbound / outbound, timestamp, operator.
- **Multi-tenant isolation** — an organisation only ever sees its own data. No leaks. We will try to break this.

3 — Analytics (BigQuery)

The dashboard reads from **BigQuery**, not Postgres. You build the sync.

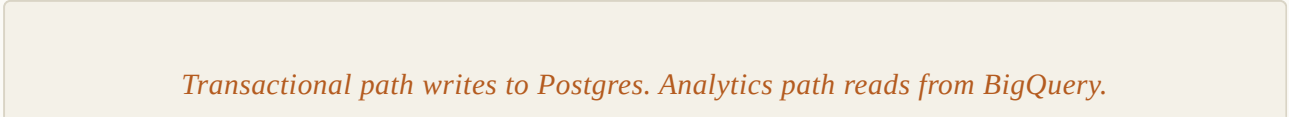
- At least **one chart** and **one data grid** showing stock levels and recent movement velocity, served from BQ.
- The sync architecture — Pub/Sub, Cloud Scheduler, dual-write, Datastream — is **your call**. Document the trade-off in the README.
- Bonus, not required: surface one anomaly. Low stock, dead stock, fast movers — whatever you find interesting.

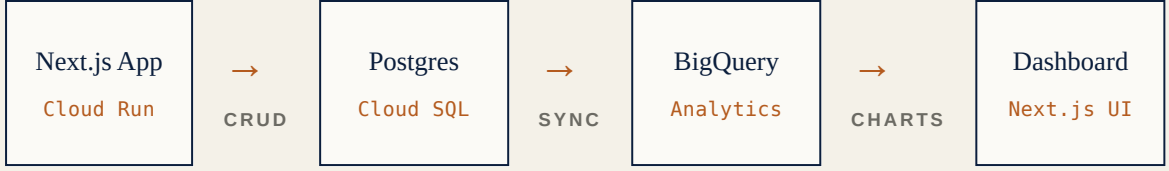
4 — One Product Call

- Propose **one feature you would add** beyond this brief, and why. Build it if you have time; describe it in the Loom if you don't.

II. The Architecture

This is the shape of the system you are building. It mirrors how we build for our enterprise client.





On the sync arrow. This is the most interesting decision in the exercise. There is no single right answer; there is a spectrum of trade-offs between latency, durability, complexity and cost. We will read your README closely for the trade-off you chose and why.

III. Stack (Required)

Everything below is required. The stack mirrors the production environment of the engagement; we cannot evaluate substitutes accurately.

Framework	Next.js 15 (App Router) · React 19 · TypeScript `strict`
Transactional DB	**Cloud SQL for PostgreSQL** via Prisma. SQLite is fine for local dev; the deployed instance must be Cloud SQL.
Analytics DB	**BigQuery** — dataset, table, IAM-scoped service account. Dashboard reads here, not Postgres.
Sync	Your choice. Cloud Scheduler + Cloud Run Job, Pub/Sub, Datastream, or a documented dual-write. Tell us why.
Auth	WorkOS AuthKit
Deploy	**Cloud Run**. Public URL. New Google Cloud accounts get $300 in free credit — comfortably enough.
Secrets	**Secret Manager** for DB credentials, WorkOS keys, BQ service account. No secrets in the repo.
Styling	Tailwind, MUI, Radix — your call.
Grid & Charts	AG Grid Community + any chart library. Mention what you would swap to in production.

A NOTE ON FRICTION

Setting up GCP, Cloud SQL, BigQuery, IAM and a Cloud Run deploy is real work — probably 90 minutes of plumbing on its own. We know.

We are *specifically* testing whether you can navigate that fluently, because the role does it weekly. If this kind of setup feels overwhelming, this engagement is probably not the right fit — and that is useful information for both of us.

IV. Dummy Data (Required)

A reviewer logging in should see *a populated system*, not an empty shell. Ship a `prisma/seed.ts` that runs against Cloud SQL and a script that mirrors a representative slice into BigQuery.

Organisations	**3 distinct orgs** — so multi-tenant isolation is genuinely testable. Realistic names: `Coastal Logistics`, `Meridian Stores`, `Tilman & Co.`
Warehouses	**~4 per org** (12 total). Distinct locations, varying capacities.
Inventory items	**~80 across the system**. Varied SKUs, quantities, distributed unevenly so analytics has shape.
Stock movements	**~400 movements** spread across the last **90 days**. Mix of inbound and outbound. Some warehouses busy, some quiet. Some SKUs fast-moving, some dead. *Make the chart interesting.*
Test users	**3 logins per org** (9 total) — one Admin, one Manager, one Operator. Credentials in the README so we can log in immediately. WorkOS sandbox / magic-link is fine.
BigQuery	The same data, denormalised appropriately for analytics. The sync should be idempotent — running it twice should not double anything.

Why this matters. Reviewers should not have to register accounts or click *Add Item* 40 times to evaluate your build. Candidates who handle this thoughtfully are the ones who think about the person on the other side of their work — exactly the instinct we hire for.

V. What to Submit

- A **working Cloud Run deploy** — public URL we can hit immediately.
- A **GitHub repo**, public or sharing access to `hello@t3c.ai`.
- A **README** that covers: how to run locally, the sync architecture you chose and why, what you cut, what you would do with another week.
- The **nine seeded logins**, listed clearly.
- A **5–7 minute Loom** walkthrough:
  - What you built and what you cut.
  - How RBAC actually enforces — show us the code path, not the UI.
  - The Postgres ↔ BigQuery sync, and the trade-off you accepted.
  - Honestly: where you used AI tools, where you overrode them, where they led you astray.

ON AI TOOLS

We **expect** you to use Claude Code, Cursor, or whatever you reach for. This is how we build at T3C and how you would build on this engagement. The Loom should show how you *orchestrated* them — where they helped, where they hallucinated, where you overrode them.

A candidate who hand-types 800 lines of React to look "pure" is a worse signal than one who shipped twice as much with AI and can explain every decision.

VI. How We'll Evaluate

What we look for

- Clean architecture and sensible trade-offs
- RBAC enforced at the data layer — actually multi-tenant
- A defensible PG → BQ sync, with the reasoning in the README
- Idempotent seeds; a system reviewers can poke around immediately
- Taste in what you chose to build versus cut
- An honest walkthrough, including weaknesses
- Effective use of AI tooling

What we don't

- Every feature complete or polished
- Pixel-perfect UI
- Made-up complexity to look impressive
- An empty system that requires registration to evaluate
- A defensive walkthrough about the worst part of your code
- Hand-coded purity for its own sake
- Refusing AI tools to seem "authentic"

VII. Interview Process

i	Intro	30 min. Who you are, what you want next, what we're building. Both ways.
ii	Build exercise	This document. 6–8 hours, within a 7-day window.
iii	Walkthrough	45 min. You walk us through the code, the architecture, the trade-offs you made.
iv	Founders' round	45 min. The product, the engagement, the team. Depth and fit.
v	Offer	Compensation, notice period, start date, paperwork.

Pace. We move quickly — typically 7 to 10 days end to end from intro to offer, assuming the build lands cleanly. We respect that you're employed elsewhere and structure rounds around that.

VIII. A Few Honest Notes

We've sent and received enough of these exercises to know the failure modes. Read these before you start — they will save you time.

If you can't finish, submit what you have.

A scoped, deployed, half-done system with a clear Loom beats a polished local prototype every time. We grade ruthlessly on what's deployed and explainable, not on what's promised.

If the brief is ambiguous, make a call and document it.

Real product work has ambiguity. We want to see how you resolve it. A README sentence — *"I assumed X because Y; the alternative would have been Z"* — is one of the strongest signals you can send.

If you're stuck on plumbing, write to us.

Don't burn three hours on an IAM config issue when a one-line answer would unblock you. We'd rather you spend the time on the architecture than on Cloud SQL connection strings. **hello@t3c.ai** — we answer fast.

If something in this brief feels wrong, push back.

A bad trade-off, a missing constraint, a better way to scope it — tell us. We'd rather hire someone who argues with the brief than someone who nods along with it. Reply to the email; we read everything.

If GCP cost is a concern, it shouldn't be.

New Google Cloud accounts get $300 of free credit. Cloud SQL on `db-f1-micro`, BigQuery sandbox tier, Cloud Run on default settings — the entire exercise costs well under $5 in real terms, and zero if you're on the free credit. Tear it down after we evaluate.

On T3C.

T3C Technologies is an AI-native product studio based in Chennai. We design, build and operate software for clients in India and the United States. We are not an offshore body shop. Our engineers own outcomes, ship pragmatically, and use AI tooling as a multiplier rather than a substitute for thinking.

This role is a dedicated engagement on a multi-tenant SaaS platform with a small, sharp engineering team. If that shape of work — high ownership, US overlap, deep technical surface, AI-first delivery — is what you want next, we'd genuinely love to see what you build.

Questions: hello@t3c.ai · **Web:** t3c.ai